

Homework 2: Implement calibrator for Tree. Part 1: Background

Remark. The results used are stated in boxes and cited by number together with the displayed equation. Each one is a result of the lecture on the binomial tree (notebooks `Tree.ipynb` and `Binomial_tree_coding.ipynb`; the cell is given in the box), a result of Homework 1, or it is proved right after its box. The code of the lecture is used unchanged; the new code is short and marked as such. All the code of this part is in the notebook `2.HW_EXE.1` (same order as here), and in the files `prof_tree.py` (lecture code), `bg_functions.py` (new code) and `plotter.py` (Homework 1). The running example is the one of the lecture: $S_0 = K = 1$, $T = 1$ year, one-year discount factor 0.95.

The assignment.

Background: In the Black–Scholes model volatility is a key parameter that determines option price. In a Balanced Binomial Tree (i.e. risk-neutral up step probability = 0.5, risk-neutral down step probability = 0.5) the same role is played by step size: higher step is like higher volatility, it results in higher option prices. Also, there is a 1-1 correspondence between step size and price if all other parameters are fixed. This part checks the background, which is what makes the calibration of the Task possible. It contains three claims:

- **(C1)** in a balanced tree the step size plays the role of the volatility;
- **(C2)** a higher step gives a higher call price;
- **(C3)** with all other parameters fixed, step size and price determine each other (1–1).

The Task (the `calibrate` method) will be part 2.

Results used.

Definition 1 (Binomial tree and risk-neutral price; `Tree.ipynb`, cells 17, 19, 33, 37–38).

In one step the stock goes from S_0 to $S_1(U) = uS_0$ or $S_1(D) = dS_0$, and money in the money market grows by the factor $1 + r$, with

$$d < 1 + r < u.$$

The risk-neutral probabilities are

$$\tilde{p} = \frac{1 + r - d}{u - d}, \quad \tilde{q} = 1 - \tilde{p}, \quad (1)$$

the price of an option with payoff V_1 is

$$V_0 = \frac{1}{1 + r} \tilde{E}(V_1) = \frac{1}{1 + r} (\tilde{p} V_1(U) + \tilde{q} V_1(D)), \quad (2)$$

and in a tree with N steps (spot tree $S_{t,i}$, $t = 0, \dots, N$, $i = 0, \dots, t$)

$$V_0 = \frac{1}{(1 + r)^N} \tilde{E}(V_N). \quad (3)$$

In the code of `Binomial_tree_coding.ipynb` (cell 8) the parameter is the discount factor of one step, $DF = 1/(1 + r)$; the leaves are multiplied by DF^N and the backward steps use $\tilde{p}$ and $\tilde{q}$.

Definition 2 (Balanced tree; `Tree.ipynb`, cell 56; `Binomial_tree_coding.ipynb`, cells 12–14).

A binomial tree is *balanced* if

$$\tilde{p} = \tilde{q} = \frac{1}{2}. \quad (4)$$

The *up step* is the factor u (`spot_mult_up`); in the balanced tree the down step d (`spot_mult_down`) is computed from u by `calcBalancedDownStep`.

Definition 3 (Flat interest rate; `Binomial_tree_coding.ipynb`, cell 20).

For a maturity of T years and N steps, each step is $\Delta t = T/N$ years long, and the discount factor of one step is the one-year discount factor DF_1 to the power Δt :

$$DF = DF_1^{\Delta t}, \quad \Delta t = \frac{T}{N}. \quad (5)$$

Then $DF^N = DF_1^T$ for every N : “discount factors have to be scaled to keep interest rate fixed”.

Theorem 1 (Vega of the Black–Scholes call; Homework 1, HW 2/2 (Lesson 3, cell 49)).

The Black–Scholes call price c has

$$\mathcal{V} = \frac{\partial c}{\partial \sigma} = S_0 N'(d_1) \sqrt{T} > 0, \quad (6)$$

so c is strictly increasing in the volatility σ . In Homework 1 the pricer `black_scholes_eur_call` (HW 2/2, new version) returns c and, with `return_greeks=True`, also $\mathcal{V}$.

The code of the lecture.

Cells 3, 5, 8 and 13 of `Binomial_tree_coding.ipynb`, unchanged (with the imports of cell 1):

lecture code (prof.tree.py)

```
import numpy as np
from scipy.optimize import minimize
from collections.abc import Callable

def european_call_payoff(S: float, K: float) -> float:
    return max(S-K, 0.0)

def create_spot_tree(spot: float, spot_mult_up: float, spot_mult_down: float, steps: int) -> list[list[float]]:
    previous_level = [spot]
    tree = [previous_level]
    for _ in range(steps):
        new_level = [s * spot_mult_down for s in previous_level]
        new_level += [previous_level[-1] * spot_mult_up]
        tree += [new_level]
        previous_level = new_level
    return tree

def create_discounted_price_tree(spot_tree: list[list[float]], discount_factor: float, K: float, diag: int = 0) -> list[list[float]]:
    spot = spot_tree[0][0]
    spot_mult_up = spot_tree[1][-1]
    spot_mult_down = spot_tree[1][0]
    p_up = ((1 / discount_factor - spot_mult_down) /
            (spot_mult_up - spot_mult_down))
    p_down = 1 - p_up
    steps = len(spot_tree) - 1
    continuation_value_tree = [[np.nan for _ in level] for level in spot_tree]
    if diag > 0:
        print("risk-neutral measure: ")
        print('%0.3f' % p_up, '%0.3f' % p_down)
        # init delta tree
        delta_tree = [[np.nan for _ in level] for level in spot_tree[:-1]] #delta makes no sense for leaves
    # going backwards, payoff is known in leaves
    for i in range(len(spot_tree[-1])):
        spot = spot_tree[-1][i]
        discounted_continuation_value = discount_factor**(steps) * european_call_payoff(spot, K)
        continuation_value_tree[-1][i] = discounted_continuation_value
    for step in range(steps - 1, -1, -1):
        for i in range(len(spot_tree[step])):
            continuation_value_tree[step][i] = p_up * continuation_value_tree[step + 1][i] + \
                p_down * continuation_value_tree[step + 1][i + 1]
            if diag > 0:
                delta_tree[step][i] = ((continuation_value_tree[step + 1][i] - continuation_value_tree[step + 1][i + 1])
                    / (spot_tree[step + 1][i] - spot_tree[step + 1][i + 1]))
    if diag > 0:
        print("delta: ")
        delta_tree_readable = [['%0.3f' % e for e in n] for n in delta_tree]
        print(delta_tree_readable)
    return continuation_value_tree

def calcBalancedDownStep(spot_mult_up: float, discount_factor: float) -> (float, float):
    return spot_mult_up - 2 * (spot_mult_up - 1 / discount_factor)
```

Lemma 1 (Balanced down step; from Definitions 1 and 2).

The tree is balanced, (4), exactly when

$$d = \frac{2}{DF} - u, \quad (7)$$

which is what `calcBalancedDownStep` returns: $u - 2(u - 1/DF) = 2/DF - u$. The condition $d < 1 + r < u$ of Definition 1 then means $u > 1/DF$, and $d > 0$ means $u < 2/DF$; so the up step can be any

$$u \in \left(\frac{1}{DF}, \frac{2}{DF} \right).$$

Proof. With $1 + r = 1/DF$, we set $\tilde{p}$ of (1) equal to $\frac{1}{2}$ and solve for d by levels:

$$\frac{1/DF - d}{u - d} = \frac{1}{2} \quad (\text{Definition 1, (1); Definition 2, (4)})$$

$$2\left(\frac{1}{DF} - d\right) = u - d \quad (\text{multiply by } 2(u - d) > 0)$$

$$d = \frac{2}{DF} - u. \quad (\text{collect } d)$$

Then $d < 1/DF$ is $2/DF - u < 1/DF$, that is $u > 1/DF$; and $d > 0$ is $u < 2/DF$. □

New code.

One function prices the call in a balanced tree with the lecture code; the second one gives the up step for a chosen *tree volatility* (introduced in (C1) below):

new code (bg_functions.py)

```
def balanced_tree_call_price(spot: float, strike: float, maturity: float, steps: int,
                             df_annual: float, spot_mult_up: float) -> float:
    """Price of a European call in a Balanced Binomial Tree (p_up = p_down = 0.5)."""
    discount_factor = df_annual ** (maturity / steps) # per step: keeps the rate flat
    spot_mult_down = calcBalancedDownStep(spot_mult_up, discount_factor)
    spot_tree = create_spot_tree(spot, spot_mult_up, spot_mult_down, steps)
    price_tree = create_discounted_price_tree(spot_tree, discount_factor, strike)
    return price_tree[0][0]

def up_step_from_tree_vol(sigma_tree: float, maturity: float, steps: int,
                           df_annual: float) -> float:
    """Up step u with step size u - 1/DF = sigma_tree * sqrt(dt)."""
    dt = maturity / steps
    discount_factor = df_annual ** dt
    return 1 / discount_factor + sigma_tree * np.sqrt(dt)
```

(C1) The step size plays the role of the volatility.

Lemma 2 (Mean and spread of one step of a balanced tree; from Definition 1 and Lemma 1).

In one step of a balanced tree the stock is multiplied by $X = S_{t+1}/S_t$, which is u or d with risk-neutral probability $\frac{1}{2}$ each. Then

$$\tilde{E}(X) = \frac{1}{DF}, \quad (8)$$

$$\sqrt{\tilde{E}((X - \tilde{E}X)^2)} = u - \frac{1}{DF}. \quad (9)$$

Proof. By Lemma 1, (7), $d = 2/DF - u$, so

$$\begin{aligned} \tilde{E}(X) &= \frac{1}{2}u + \frac{1}{2}d \\ &= \frac{1}{2}\left(u + \frac{2}{DF} - u\right) \\ &= \frac{1}{DF}. \end{aligned}$$

Hence $u - \tilde{E}X = u - 1/DF$ and $d - \tilde{E}X = 1/DF - u = -(u - 1/DF)$: both values are at the same distance $u - 1/DF$ from the mean, and

$$\begin{aligned} \tilde{E}((X - \tilde{E}X)^2) &= \frac{1}{2}\left(u - \frac{1}{DF}\right)^2 + \frac{1}{2}\left(u - \frac{1}{DF}\right)^2 \\ &= \left(u - \frac{1}{DF}\right)^2. \end{aligned}$$

Since $u > 1/DF$ (Lemma 1), the square root is $u - 1/DF$. □

So in a balanced tree the mean of a step is fixed by the interest rate, (8), and the up step only changes the spread around it, (9). We call

$$h = u - \frac{1}{DF}$$

the *step size*: it is the standard deviation of one step, which in the Black–Scholes model is about $\sigma\sqrt{\Delta t}$. To compare trees with different numbers of steps with each other and with σ , we use the *tree volatility*

$$\sigma_{\text{tree}} = \frac{h}{\sqrt{\Delta t}} = \frac{u - 1/DF}{\sqrt{\Delta t}}, \quad \text{that is,} \quad u = \frac{1}{DF} + \sigma_{\text{tree}}\sqrt{\Delta t}$$

(function `up_step_from_tree_vol`). The lecture (cell 20) chooses the up step from a volatility σ as $u = \exp((r - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t})$; for $\sigma = 0.1$ and 7 steps per year this gives $\sigma_{\text{tree}} \approx 0.1002$, almost the same number. The check:

check of (C1)

```
# (C1) one step of a balanced tree: mean and standard deviation of the factor S_{n+1}/S_n
discount_factor = 0.95
for spot_mult_up in [1.10, 1.20, 1.40]:
    spot_mult_down = calcBalancedDownStep(spot_mult_up, discount_factor)
    p_up = (1 / discount_factor - spot_mult_down) / (spot_mult_up - spot_mult_down)
    mean = p_up * spot_mult_up + (1 - p_up) * spot_mult_down
    std = np.sqrt(p_up * (spot_mult_up - mean)**2 + (1 - p_up) * (spot_mult_down - mean)**2)
    print(f"u = {spot_mult_up:.2f}  d = {spot_mult_down:.4f}  p_up = {p_up:.2f}  "
          f"mean = {mean:.4f}  std = {std:.4f}  u - 1/DF = {spot_mult_up - 1/discount_factor:.4f}")

# the up step of the lecture (cell 20): sigma = 0.1, 7 steps per year
steps_per_year = 7
dt = 1 / steps_per_year
discount_factor = 0.95 ** dt
spot_mult_up = np.exp((0.05 - 0.5 * 0.1 ** 2) * dt + 0.1 * np.sqrt(dt))
print(f"lecture: u = {spot_mult_up:.6f}  step size u - 1/DF = {spot_mult_up - 1/discount_factor:.6f}"
      f"  sigma_tree = {(spot_mult_up - 1/discount_factor) / np.sqrt(dt):.4f}")
```

output

```
u = 1.10  d = 1.0053  p_up = 0.50  mean = 1.0526  std = 0.0474  u - 1/DF = 0.0474
u = 1.20  d = 0.9053  p_up = 0.50  mean = 1.0526  std = 0.1474  u - 1/DF = 0.1474
u = 1.40  d = 0.7053  p_up = 0.50  mean = 1.0526  std = 0.3474  u - 1/DF = 0.3474
lecture: u = 1.045218  step size u - 1/DF = 0.037863  sigma_tree = 0.1002
```

In every case $\tilde{p} = 0.5$, the mean is $1/DF \approx 1.0526$ and the standard deviation equals $u - 1/DF$, as Lemma 2 says.

(C2) and (C3) for a one-step tree.

Lemma 3 (Price of a call in a one-step balanced tree; from Definition 1 and Lemma 1).

Let $N = 1$, $u \in (1/DF, 2/DF)$, and let the strike satisfy $0 < K < 2S_0/DF$. Then the call price is

$$V_0(u) = \frac{DF}{2} \left((uS_0 - K)^+ + \left(\left(\frac{2}{DF} - u \right) S_0 - K \right)^+ \right). \quad (10)$$

With $u^* = \max\{K/S_0, 2/DF - K/S_0\}$, the price is constant, equal to $(S_0 - DF K)^+$, for $u \leq u^*$, and

$$V_0(u) = \frac{DF}{2} (uS_0 - K) \quad \text{for } u^* \leq u < \frac{2}{DF},$$

which is strictly increasing in u . Hence $u \mapsto V_0(u)$ is a bijection

$$\left[u^*, \frac{2}{DF} \right) \longrightarrow \left[(S_0 - DF K)^+, S_0 - \frac{DF K}{2} \right). \quad (11)$$

Proof. By Definition 1, (2), with $\tilde{p} = \tilde{q} = \frac{1}{2}$, $1/(1+r) = DF$, the call payoffs $V_1(U) = (uS_0 - K)^+$, $V_1(D) = (dS_0 - K)^+$ and $d = 2/DF - u$ (Lemma 1, (7)) we get (10). Now $uS_0 > K$ exactly when $u > K/S_0$, and $dS_0 < K$ exactly when $u > 2/DF - K/S_0$.

- If $u \geq u^*$: the up leaf is at or above K and the down leaf at or below K , so $V_0 = \frac{DF}{2}(uS_0 - K)$, a line with slope $DF S_0/2 > 0$.

- If $u \leq u^*$: then $uS_0 \leq K$ or $dS_0 \geq K$, and since $dS_0 < uS_0$, both leaves are on the same side of K . If both are at or above K , so is their mean $\frac{u+d}{2}S_0 = S_0/DF$, hence $S_0 - DF K \geq 0$, and by $u + d = 2/DF$

$$V_0 = \frac{DF}{2}(uS_0 + dS_0 - 2K) = \frac{DF}{2}\left(\frac{2S_0}{DF} - 2K\right) = S_0 - DF K.$$

If both are at or below K , then $S_0/DF \leq K$ and $V_0 = 0$. In both cases $V_0 = (S_0 - DF K)^+$, which does not depend on u .

The value at u^* is $(S_0 - DF K)^+$ from both formulas, and the line tends to $\frac{DF}{2}\left(\frac{2S_0}{DF} - K\right) = S_0 - \frac{DF K}{2}$ as $u \rightarrow 2/DF$. A continuous, strictly increasing function on an interval is a bijection onto the interval between its end values, which gives (11). $\square$

The check computes the price with the lecture code for 400 up steps u and three strikes (in the money, at the money forward $K = S_0/DF$ as in cell 16 of the lecture, out of the money), compares it with (10), and draws Figure 1 with the plotter of Homework 1 (HW 2/1, `plotter.py`):

check of (C2) and (C3), one step

```
# (C2), (C3) one-step tree: price against the up step u, for three strikes
spot, maturity, df_annual = 1.0, 1.0, 0.95
u_grid = np.linspace(1 / df_annual + 1e-6, 2 / df_annual - 1e-6, 400)
strikes = [0.9, spot / df_annual, 1.2] # in the money, ATM forward, out of the money
prices = [[balanced_tree_call_price(spot, K, maturity, 1, df_annual, u) for u in u_grid]
          for K in strikes]

# the formula of Lemma 3 gives the same prices
for K, p in zip(strikes, prices):
    d_grid = 2 / df_annual - u_grid
    formula = df_annual / 2 * (np.maximum(u_grid * spot - K, 0) + np.maximum(d_grid * spot - K, 0))
    u_star = max(K / spot, 2 / df_annual - K / spot)
    print(f"K = {K:.4f}: max |tree - formula| = {np.max(np.abs(np.array(p) - formula)):.1e}, "
          f"u* = {u_star:.4f}, price range [{p[0]:.4f}, {p[-1]:.4f}]")

fig, ax = my_plotter(u_grid, prices,
                     layout={"title": "One-step balanced tree: call price against the up step",
                              "x_label": "up step $u$", "y_label": "call price", "hline": None,
                              "grid": True, "figsize": (8, 4), "title_fontsize": 13,
                              "label_fontsize": 11, "legend_fontsize": 10, "linewidth": 2},
                     names=["K = 0.9", "K = 1/0.95 (ATM forward)", "K = 1.2"])
```

output

```
K = 0.9000: max |tree - formula| = 2.8e-17, u* = 1.2053, price range [0.1450, 0.5725]
K = 1.0526: max |tree - formula| = 0.0e+00, u* = 1.0526, price range [0.0000, 0.5000]
K = 1.2000: max |tree - formula| = 0.0e+00, u* = 1.2000, price range [0.0000, 0.4300]
```

The lecture code and (10) agree to rounding; u^* and the price range are the ones of Lemma 3: for $K = 0.9$, $u^* = 2/0.95 - 0.9 \approx 1.2053$ and the range is $[1 - 0.95 \cdot 0.9, 1 - 0.95 \cdot 0.45] = [0.1450, 0.5725]$.

(C2) and (C3) for several steps, and the comparison with Black–Scholes.

For $N = 1, 2, 5, 50$ steps we price the call $S_0 = K = 1$, $T = 1$ with the flat rate of Definition 3 (one-year discount factor 0.95), for tree volatilities from 0.005 to 0.6. Next to it, the Black–Scholes price of Homework 1 with the same rate, $r = -\ln 0.95$, and $\sigma = \sigma_{\text{tree}}$:

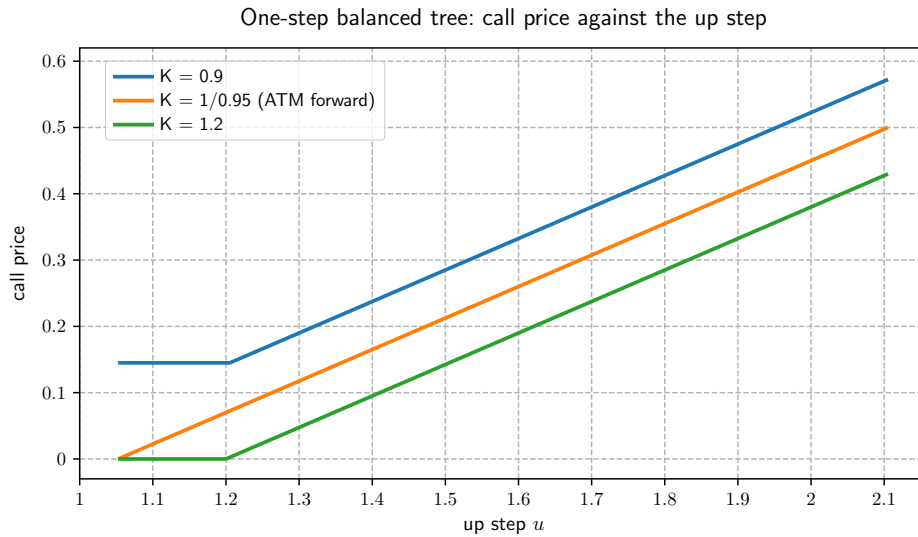


Figure 1: Output of the one-step check ($S_0 = 1$, $T = 1$, $DF = 0.95$). Each price is flat up to u^* and then increases linearly; for the at-the-money-forward strike $u^* = 1/DF$, so it increases on the whole range.

check of (C2) and (C3), several steps

```
# (C2), (C3) N-step trees against the tree volatility, and the Black-Scholes price of Homework 1
spot, strike, maturity, df_annual = 1.0, 1.0, 1.0, 0.95
r = -np.log(df_annual) # the same flat interest rate
sigma_grid = np.linspace(0.005, 0.6, 120)
curves, names = [], []
for steps in [1, 2, 5, 50]:
    curves.append([balanced_tree_call_price(spot, strike, maturity, steps, df_annual,
                                             up_step_from_tree_vol(s, maturity, steps, df_annual))
                  for s in sigma_grid])
    names.append(f"tree, N = {steps}")
curves.append([black_scholes_eur_call(r=r, T=maturity, S0=spot, sigma=s, K=strike)[0]
              for s in sigma_grid]) # one sigma at a time (assert sigma > 0)
names.append("Black-Scholes (Homework 1)")

for s in [0.1, 0.2, 0.3]:
    i = np.argmin(np.abs(sigma_grid - s))
    print(f"sigma = {sigma_grid[i]:.3f}: " + " ".join(f"{c[i]:.4f}" for c in curves))
print("increasing:", [bool(np.all(np.diff(c) >= -1e-12)) for c in curves])

fig, ax = my_plotter(sigma_grid, curves,
                    layout={"title": "Call price against the tree volatility (S0 = K = 1, T = 1)",
                            "x_label": "tree volatility $(u - 1/DF)/\\sqrt{\\Delta t}$, or $\\sigma$",
                            "y_label": "call price", "hline": None, "grid": True, "figsize": (8, 4),
                            "title_fontsize": 13, "label_fontsize": 11, "legend_fontsize": 10,
                            "linewidth": 2, "linestyles": ["-", "-", "-", "-", "--"],
                            "colors": ["tab:blue", "tab:orange", "tab:green", "tab:red", "black"]},
                    names=names)
```

output (columns: N = 1, 2, 5, 50, Black-Scholes)

```
sigma = 0.100: 0.0725 0.0708 0.0671 0.0690 0.0689
sigma = 0.200: 0.1200 0.1017 0.1076 0.1055 0.1052
sigma = 0.300: 0.1675 0.1302 0.1480 0.1426 0.1429
increasing: [True, True, True, True, True]
```

All five curves of Figure 2 are increasing (the line `increasing`), and the tree with 50 steps is already close to the Black-Scholes price: 0.0690 against 0.0689 at $\sigma = 0.1$, 0.1055 against 0.1052 at $\sigma = 0.2$. The Black-Scholes price is strictly increasing in σ by Theorem 1, (6); the tree prices behave in the same way in the step size, which is the meaning of (C1)–(C3).

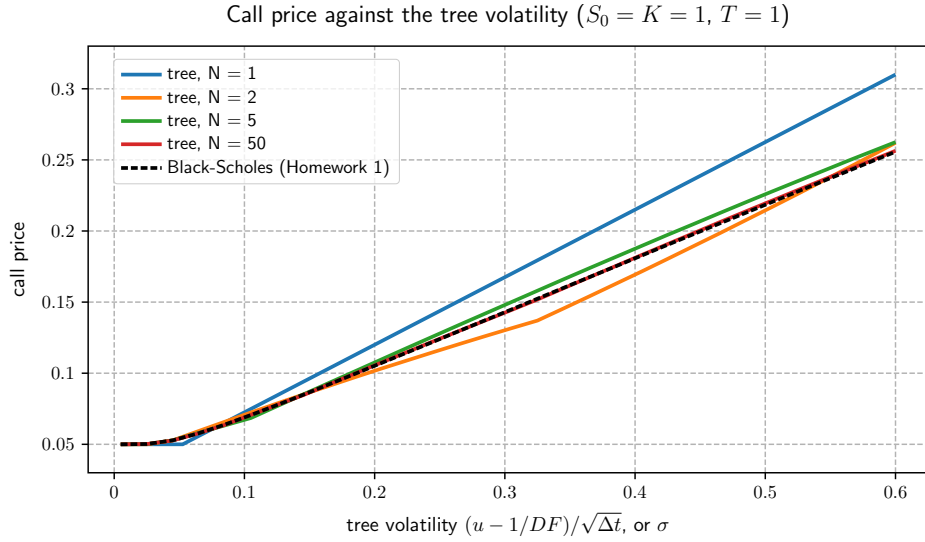


Figure 2: Output of the check with several steps. The tree prices increase with the step size like the Black-Scholes price increases with σ , and approach it as N grows.

Hence the background holds, with the step size $h = u - 1/DF$ in the role of the volatility:

- **(C1)** in a balanced tree $d = 2/DF - u$ (Lemma 1); one step has mean $1/DF$, fixed by the interest rate, and standard deviation $u - 1/DF$ (Lemma 2), like $\sigma\sqrt{\Delta t}$ in Black-Scholes;
- **(C2)** the call price does not decrease when the up step increases (Lemma 3 for one step, Figures 1 and 2), as the Black-Scholes price increases with σ (Theorem 1);
- **(C3)** once the strike lies between the leaves, the price is strictly increasing and continuous in u , so each attainable price comes from exactly one up step; for one step, (11): $u \in [u^*, 2/DF)$ and $V_0 \in [(S_0 - DF K)^+, S_0 - DF K/2)$.

Comment (to keep in mind for the Task). The 1–1 correspondence needs two conditions. First, the price is flat for small steps, as long as all leaves are on the same side of the strike (for one step, $u \leq u^*$); there the price is the lower bound $(S_0 - DF^N K)^+$, and every u in the flat part gives it. Second, not every price can be reached: $d > 0$ bounds the up step by $2/DF$, so for one step the price stays below $S_0 - DF K/2$. So the calibration of part 2 must check that the given price lies in the attainable range and must look for u above the flat part; with more steps the flat part is very short (Figure 2).